

FORMATION EMBARQUÉE

Antisèche — C++ Embarque

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Antisèche — C++ embarqué

Le « C++ embarqué raisonnable » : classes, RAII, templates, `constexpr`, références. **Sans** exceptions, RTTI, ni allocation dynamique imprévisible (`-fno-exceptions -fno-rtti`).

Ce que C++ ajoute tout de suite

```
bool ok = true;           // vrai booléen
int& ref = n;             // référence : alias toujours valide
void f(const Measure& m);  // passage sans copie
auto x = 42u;             // déduction de type
constexpr uint32_t F = 16'000'000; // calculé à la COMPILE
static_assert(F % 1000 == 0, "message"); // vérifié à la compilation
enum class Mode : uint8_t { Repos, Actif }; // typée, pas de fuite de noms
nullptr                  // au lieu de NULL
```

Classe type

```
class Led {
public:
    explicit Led(uint8_t broche) : broche_(broche) { // liste d'init
        pinMode(broche_, OUTPUT);
    }
    ~Led() { eteindre(); } // destructeur

    void allumer() { digitalWrite(broche_, HIGH); on_ = true; }
    bool estAllumee() const { return on_; } // const = ne modifie rien

    Led(const Led&) = delete; // interdire la copie
    Led& operator=(const Led&) = delete;

private:
    uint8_t broche_;
    bool on_ = false; // init par défaut
};
```

RAII — l'idée maîtresse

Ressource acquise dans le **constructeur**, libérée dans le **destructeur** : libération **garantie** sur tous les chemins de sortie (return anticipé compris).

```

class SectionCritique {
public:
    SectionCritique() { noInterrupts(); }
    ~SectionCritique() { interrupts(); }
    SectionCritique(const SectionCritique&) = delete;
};

void f() {
    SectionCritique sc;          // ← IRQ coupées
    if (erreur) return;          // ← elles seront QUAND MÊME rétablies
}                                // ← ici aussi

```

Usages : section critique · chip-select SPI · mutex RTOS · transaction I2C.

Héritage & polymorphisme (drivers interchangeable)

```

class ICapteur {
public:
    virtual ~ICapteur() = default;          // OBLIGATOIRE si héritage
    virtual int16_t lire() = 0;             // = 0 → classe abstraite
};

class Bme280 : public ICapteur {
public:
    int16_t lire() override { return ...; } // override = vérifié
};

ICapteur* tab[] = { &bme, &ds18b20 };
for (ICapteur* c : tab) log(c->lire());    // la bonne méthode est appelée

```

Coût : une vtable (1 pointeur/objet + 1 indirection). L'alternative sans coût = **templates** (résolu à la compilation).

Templates

```

template <typename T, size_t N>
class Tampon {
    static_assert((N & (N-1)) == 0, "N doit être une puissance de 2");
public:
    bool put(const T& v);
    bool get(T& v);
private:
    T buf_[N] {};                // stockage STATIQUE, zéro malloc
    volatile size_t tete_ = 0, queue_ = 0;
};

Tampon<uint8_t, 64> rx;          // instancié à la compilation

template <typename T>
constexpr T borner(T v, T lo, T hi) { return v < lo ? lo : (v > hi ? hi : v); }

```

STL en embarqué

✓ Utilisable sur micro

`std::array<T,N>`

`std::optional`, `std::pair`

`<algorithm>` (`min`, `max`, `clamp`, `sort`)

`std::atomic` (si supporté)

✗ À éviter (allocation dynamique)

`std::vector`, `std::string`, `std::map`

`std::function` (peut allouer)

`iostream` (très lourd)

exceptions, RTTI

Sur Linux embarqué (Pi, i.MX) : toute la STL est utilisable.

Lambdas (callbacks)

```
bouton.surAppui([]() { led.basculer(); }); // sans capture
auto seuil = 100;
filtrer([seuil](int16_t v) { return v > seuil; }); // capture par valeur
```

Interopérer avec du C (HAL, drivers constructeur)

```
extern "C" {
#include "stm32f4xx_hal.h" // empêche le name mangling
}
extern "C" void HAL_UART_RxCpltCallback(UART_HandleTypeDef *h) { }
```

Pièges spécifiques

1. Les **objets globaux** ont leur constructeur exécuté **avant** `main()` → ne pas y toucher au matériel non initialisé.
2. `new` / `delete` : mêmes réserves que `malloc` (fragmentation).
3. Un destructeur **non virtuel** dans une classe de base = fuite/UB.
4. Copie implicite d'un objet qui détient une ressource → `= delete`.
5. `std::string` par valeur en paramètre = copie + allocation → `const&`.